



University Institute of Engineering

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Unit- 2

Subject Name : Operating System

Chapter : Memory Management- (Part-2)

Prepared By: Er. Inderjeet Singh(e8822)



Outline

- Memory Management and its Need
- Linking and Loading
- Address Binding
- Logical v/s Physical address space
- Swapping
- Types of Memory Allocation
- Fragmentation
- Paging
- Exercise on Paging

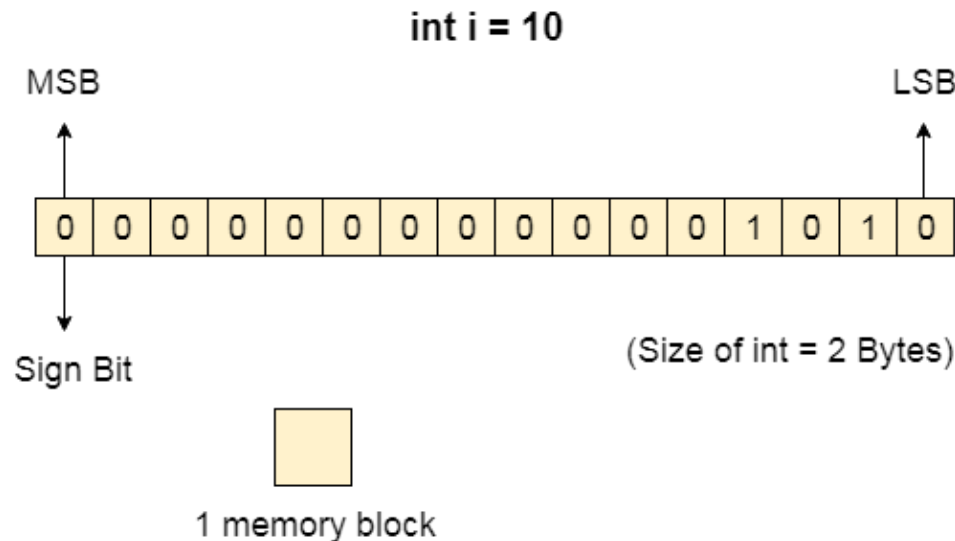
Memory Management

Memory Management is the process of controlling and coordinating computer memory, assigning portions called blocks to various running programs to optimize overall system performance.

- When the program requests a block of memory, a part of the memory manager called the allocator assigns that block to program.
- When a program no longer needs the data in previously allocated memory blocks, these blocks become available for reassignment. This task can be done automatically by memory manager or manually by the user.

Need of Memory Management

- Memory management is an essential element of all modern computing systems
- With the continued use of virtualization and the need to optimize resource utilization, memory is constantly being allocated, removed, segmented, used and re-used.
- With memory management techniques, memory management errors, that can lead to system and application instability and failures can be mitigated

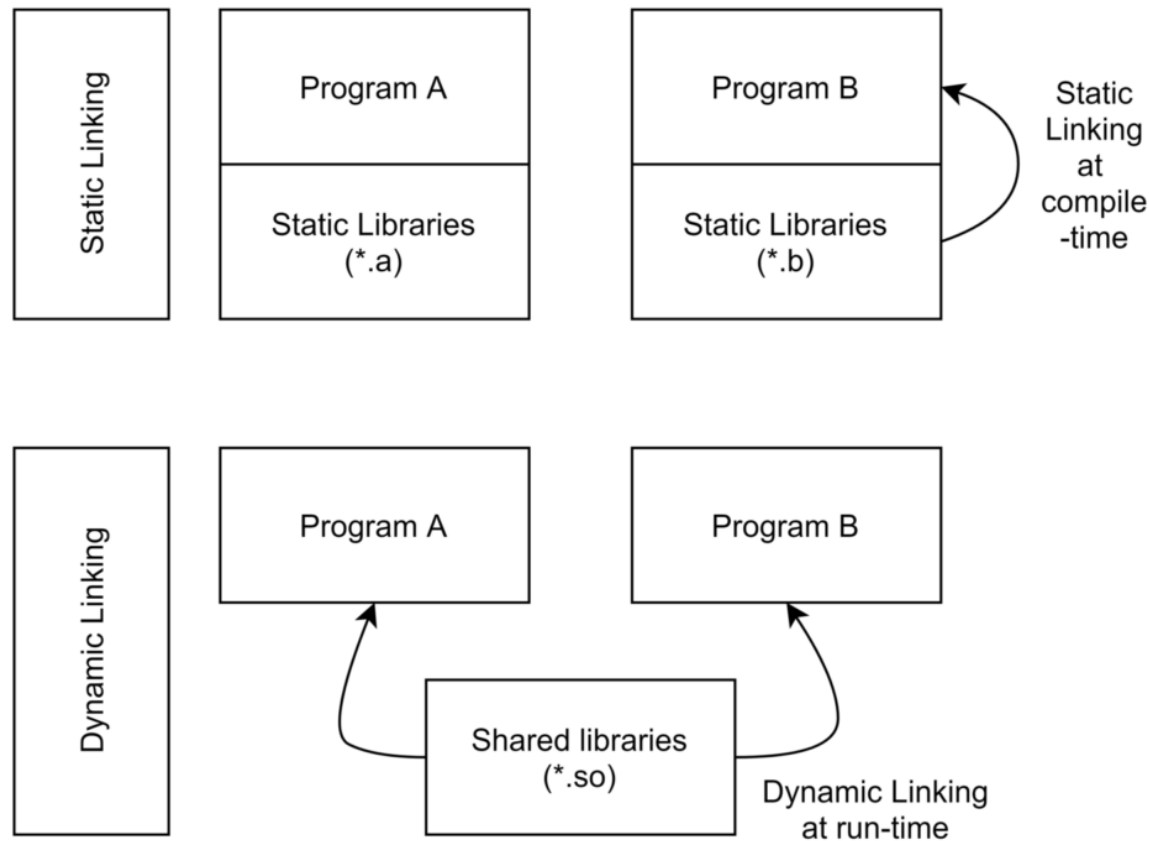




Linking

- Linking is the **process of connecting all the modules or the function of a program for program execution.**
- It takes more than one object module and combines it into a single object file.
- The linker, also known as the link editor, takes object modules from the assembler and forms an executable file for the loader. Thus, as the name suggests, linking is the process of collecting and maintaining pieces of data and code into a single file.
- Linking of object modules **is done at both compile as well as load time.** Compile-time linking is done when the source code is translated to machine code. The load-time linking is done while the program is loaded into memory by the loader.

Static and Dynamic Linking



For instance, in the above diagram, in the static linking, each program binds to its dependent libraries at compile time.

Whereas, in the case of dynamic linking, programs use shared libraries, and these libraries are linked to the programs at run time.

Static and Dynamic Linking

- Static linking is **done at the time of compilation of the program**. It takes the collection of object files and the command-line arguments and generates the fully-linked object file that is loaded and executed.
- Dynamic linking is another technique that **intends to reduce the shortcomings of static linking**. With static linking, the user ends up copying functions or routines that are repetitive across various executables. Thus, static linking becomes inefficient. For instance, nearly every program needs *printf()* function. Thus, a copy of it is present in all executables which wastes space in both virtual memory and in the file system.
- In the dynamic linking approach, the linker does not copy the routines into the executables. It takes note that the program has a dependency on the library. When the program actually executes, the **linker binds the function calls in the program to the shared library present in the disk**.



	Static Linking	Dynamic Linking
Definition	The process of combining all necessary library routines and external references into a single executable file at compile-time.	The process of linking external libraries and references at runtime, when the program is loaded or executed.
Linking Time	Occurs at compile-time.	Occurs at runtime.
File Size	Generally larger file size, as all required libraries are included in the executable.	Smaller file size, as libraries are linked dynamically at runtime.
Flexibility	Less flexible, as any updates or changes to the libraries require recompilation and relinking of the entire program.	More flexible, as libraries can be updated or replaced without recompiling the program.
Performance	Faster program startup and direct execution, as all libraries are already linked.	Slightly slower program startup due to the additional linking process, but overall performance impact is minimal.
Examples	Executables with file extensions like .exe, .elf, .a, .lib, etc.	Executables with file extensions like .dll, .so, .dylib, etc.

Loader

- Loading is the **process of loading the program from secondary memory to the main memory** for execution and **Loader** is responsible for the same.
- Dynamic loading is the technique through which a computer program at runtime load a library into memory, retrieve the variable and function addresses, executes the functions, and unloads the program from memory. It is often used to implement software plugins.
- For instance, Apache Web Server loads the dynamic shared object (*.dso) files at runtime using this technique.

Loading v/s Linking

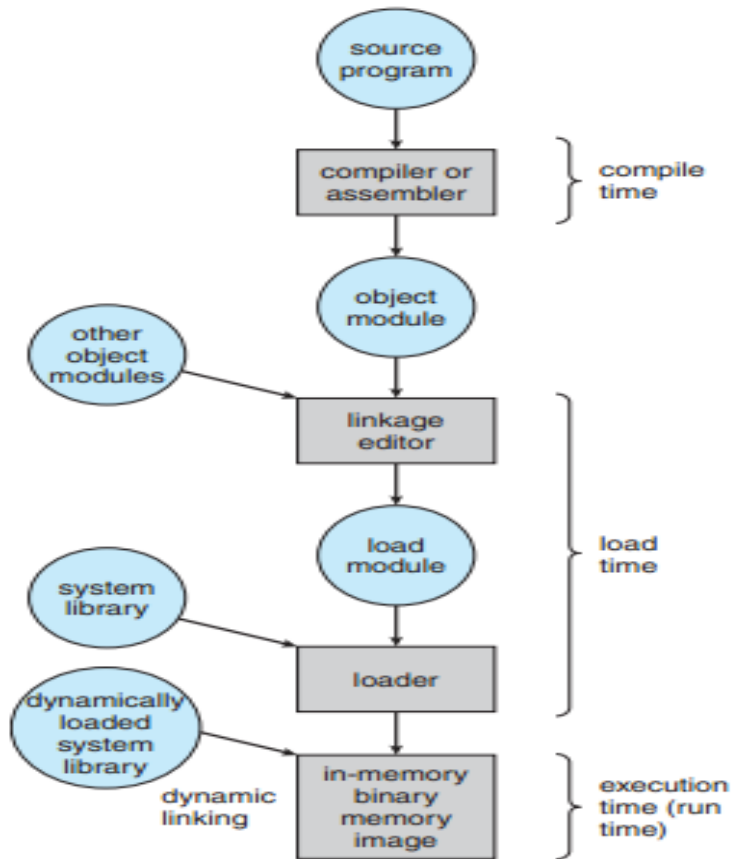


Figure 8.3 Multistep processing of a user program.

- First, an assembler or compiler translates the source program to an object module.
- A linker then uses this object module and other modules to prepare the linked module.
- A loader uses the linked modules along with other system libraries to generate the binary memory image. **Thus, the process of program execution uses the offerings of both linker and loader.**
- Generally, a program can consist of multiple modules, and based on the program execution, not all modules can be present in the memory at the same time.
- **Dynamic loading provides us the capability to load a module on demand.**
- For instance, in Java, we can use an instance of a **ClassLoader** to dynamically load a class if we need access to it. Besides, for larger applications, it is not possible to load all modules to the memory at the same time. Thus, dynamic loading is a convenient technique in such cases.

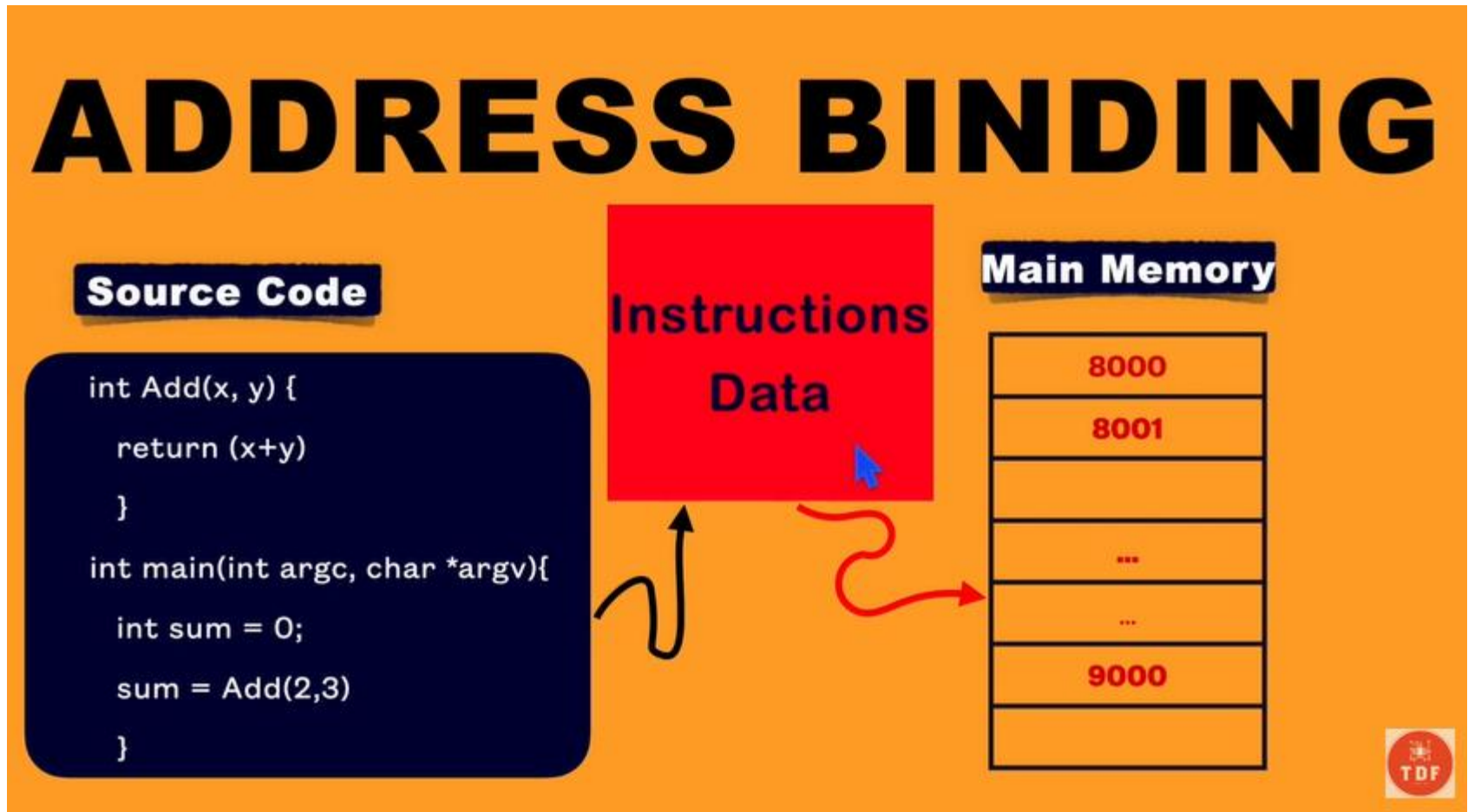
Address Binding

- Usually, a program resides on a disk as a binary executable file. To be executed, the program must be brought into memory and placed within a process. Depending on the memory management in use, the process may be moved between disk and memory during its execution.
- Address binding is the process of mapping from one address space to another address space.

Or

- When we bind data or instructions to address then it is called as Address Binding.
- Logical address is an address generated by the CPU during execution, whereas Physical Address refers to the location in the memory unit(the one that is loaded into memory).
- The logical address undergoes translation by the MMU or address translation unit in particular. The output of this process is the appropriate physical address or the location of code/data in RAM.
- Addresses in the Source program are generally symbolic.
- A compiler binds these symbolic addresses to relocatable addresses.
- The loader binds these relocatable addresses to absolute addresses.

Address Binding





Types of Address Binding

Compile Time –

If you know that during compile time, where process will reside in memory, then an absolute address is generated before loading the program. i.e. The physical address is embedded to the executable of the program during compilation.

So, this is possible only if we know, before loading and execution, how much memory to use. If that's the case, we can infer from where to start allocating memory for our program and prepare everything in advance. However, that's rarely possible. **An example of a system with compile-time address binding is MS-DOS.**

Advantage: Minimum setup time.

Disadvantage: Program to get loaded into the main memory is targeting to a memory location is occupied by any other program, then collision will occur. The current program will override.



Types of Address Binding

- **Load time – Must Generate re-locatable code if memory location is not known at compile time.** If it is not known at the compile time where the process will reside, then a **relocatable address will be generated.** The loader translates the relocatable address to an absolute address. The base address of the process in main memory is added to all logical addresses by the loader to generate an absolute address. In this, if the base address of the process changes, then we need to reload the process again.

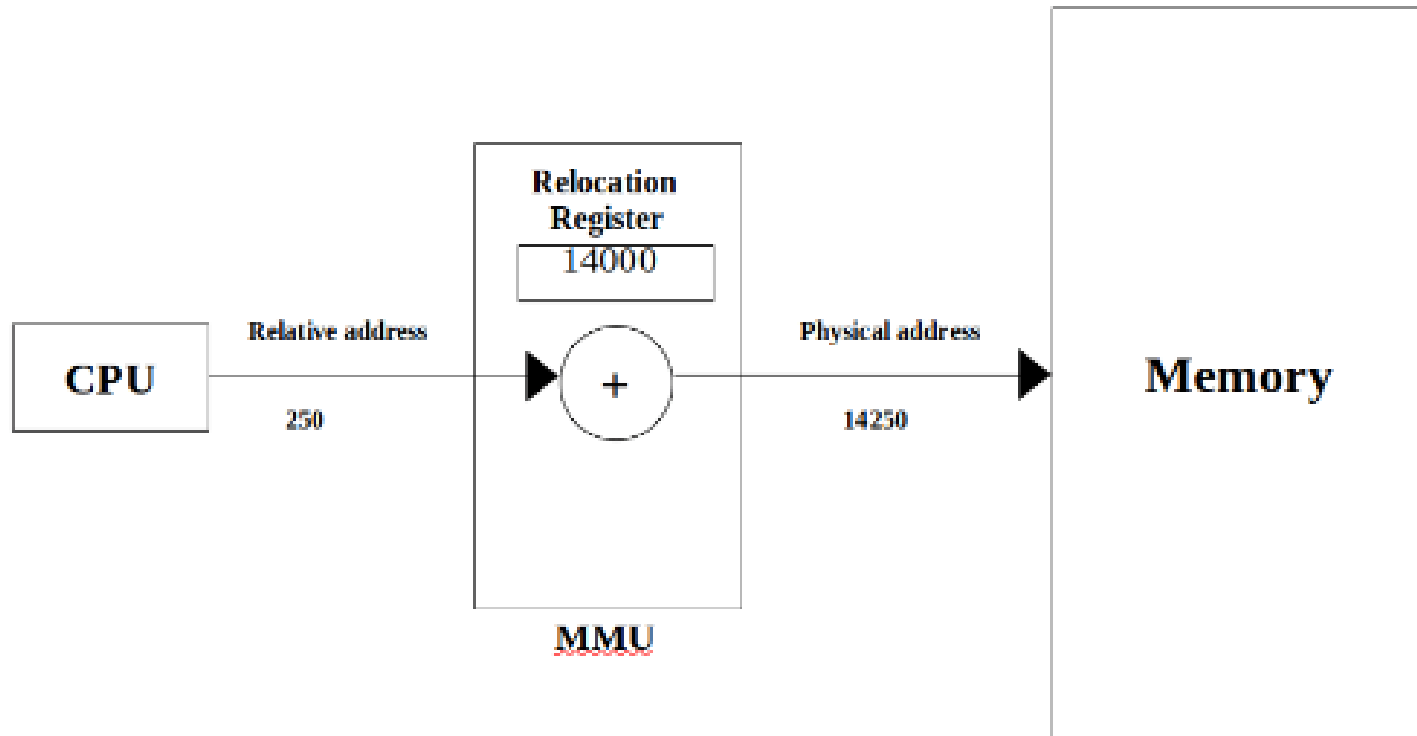
Or

- **The load-time address binding takes place during the loading of a program into the memory.** The operating system memory manager (loader) is the element in charge.
- Here, the compiler doesn't do binding but translates symbolic addresses from the source code to relative addresses.
- The loader then translates relative addresses to absolute addresses. It needs to know the base address in the main memory from which to start the allocation. Therefore, when loading a program, we compute the physical addresses by adding them to the base address:

•

Types of Address Binding

- In the image above, the relative address 250 becomes the absolute address 142500 because the base is 14000.



Types of Address Binding

- **Execution time Binding** – The instructions are in memory and are being processed by the CPU. Additional memory may be allocated and/or deallocated at this time. This is used if a process can be moved from one memory to another during execution(dynamic linking-Linking that is done during load or run time). E.g – Compaction. Or
- If the program is compiled, all the variables are known in advance. However, the memory requirements may not be if, for example, we load some data from a file dynamically, and we can't determine how much space we need for the data before we read the file. Similarly, if the program is interpreted, not all variables can be considered before execution.
- **Execution-time binding covers these cases. Addresses and memory are reserved dynamically**, i.e., when we need to do that during execution.
- The majority of general-purpose OSs can use this mechanism. Some popular OSs like Windows and Linux practically implement dynamic loading, dynamic linking, and dynamic address binding.



Mapping Virtual Addresses to Physical Addresses

- The Memory Management Unit is a combination of 2 registers –
 1. Base Register (Relocation Register)
 2. Limit Register.
- **Base Register** – contains the starting physical address of the process.
- **Limit Register** -mentions the limit relative to the base address on the region occupied by the process.

Mapping Virtual Addresses to Physical Addresses

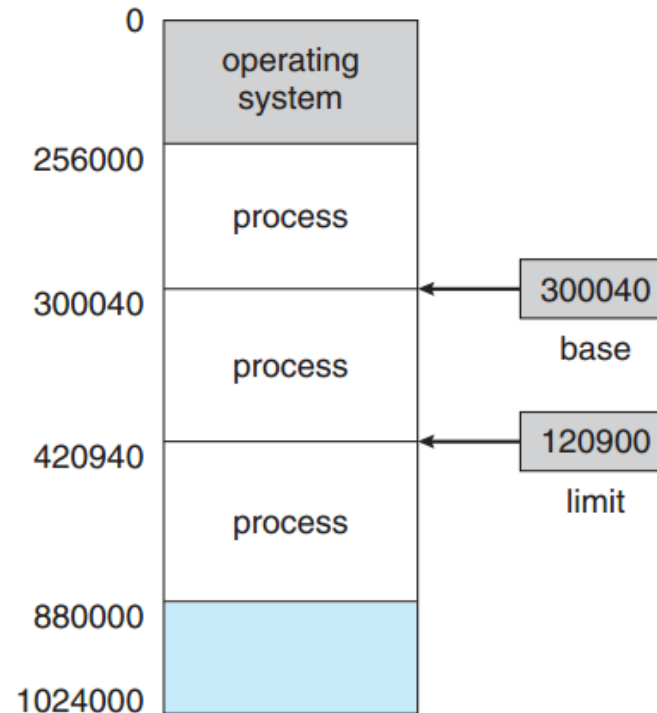


Figure 8.1 A base and a limit register define a logical address space.

Hardware Address Protection

Protection of memory space is accomplished by having the CPU hardware compare every address generated in user mode with the registers. Any attempt by a program executing in user mode to access operating-system memory or other users' memory results in a trap to the operating system, which treats the attempt as a fatal error (Figure 8.2).

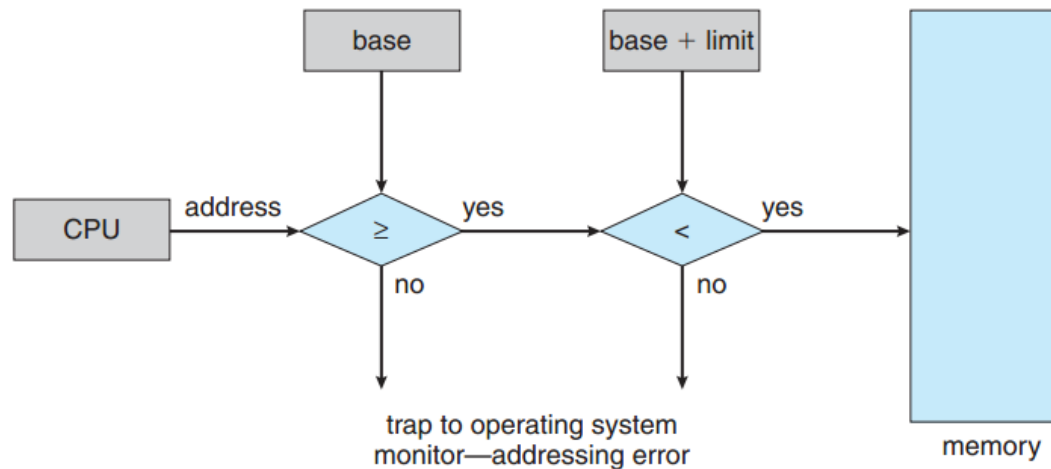


Figure 8.2 Hardware address protection with base and limit registers.

Memory Protection

- The relocation-register scheme provides an effective way to allow the operating system's size to change dynamically. This flexibility is desirable in many situations.

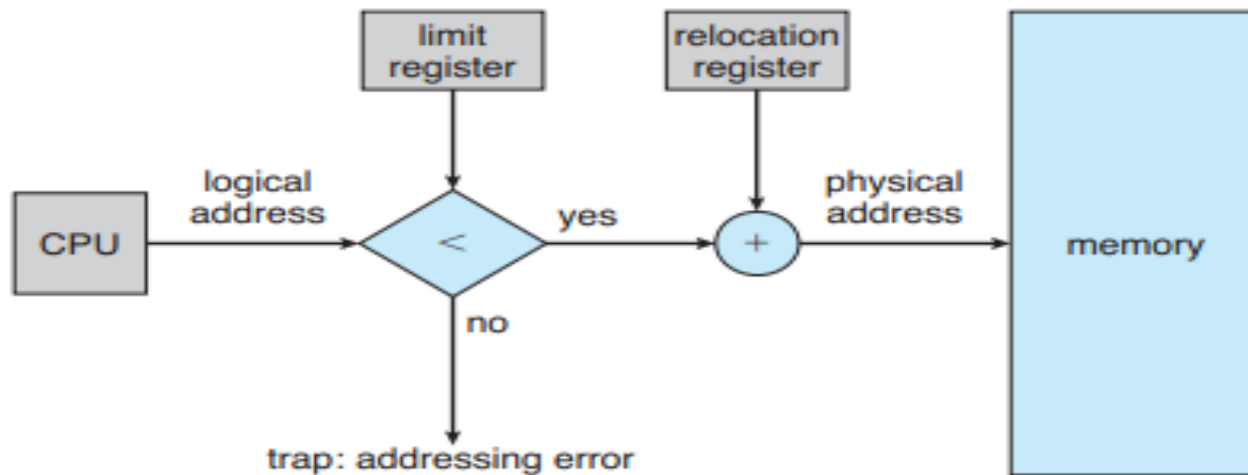


Figure 8.6 Hardware support for relocation and limit registers.

Memory Protection

- If we have a system with a relocation register, together with a limit register, we accomplish our goal. The relocation register contains the value of the smallest physical address; the limit register contains the range of logical addresses (for example, relocation = 100040 and limit = 74600). Each logical address must fall within the range specified by the limit register. The MMU maps the logical address dynamically by adding the value in the relocation register. This mapped address is sent to memory (Figure 8.6).
- When the CPU scheduler selects a process for execution, the dispatcher loads the relocation and limit registers with the correct values as part of the context switch.



Logical Versus Physical Address Space

- An address generated by the CPU is commonly referred to as a **logical address**
- An address seen by the memory unit — that is, the one loaded into the memory-address register of the memory — is commonly referred to as a **physical address**.
- The compile-time and load-time address-binding methods generate identical logical and physical addresses. However, the execution-time address-binding scheme results in differing logical and physical addresses.
- We usually refer to the logical address as a virtual address.



Logical Versus Physical Address Space

- The set of all logical addresses generated by a program is a **logical address space** .
- The set of all physical addresses corresponding to these logical addresses is a **physical address space**. Thus, in the execution-time address-binding scheme, the logical and physical address spaces differ.
- The run-time mapping from virtual to physical addresses is done by a hardware device called the **memory-management unit(MMU)**.
- We illustrate this mapping with a simple MMU scheme that is a generalization of the base-register scheme. The base register is now called a relocation register.
- The value in the relocation register is added to every address generated by a user process at the time the address is sent to memory (see Figure 8.4)

Logical Versus Physical Address Space(Contd.)

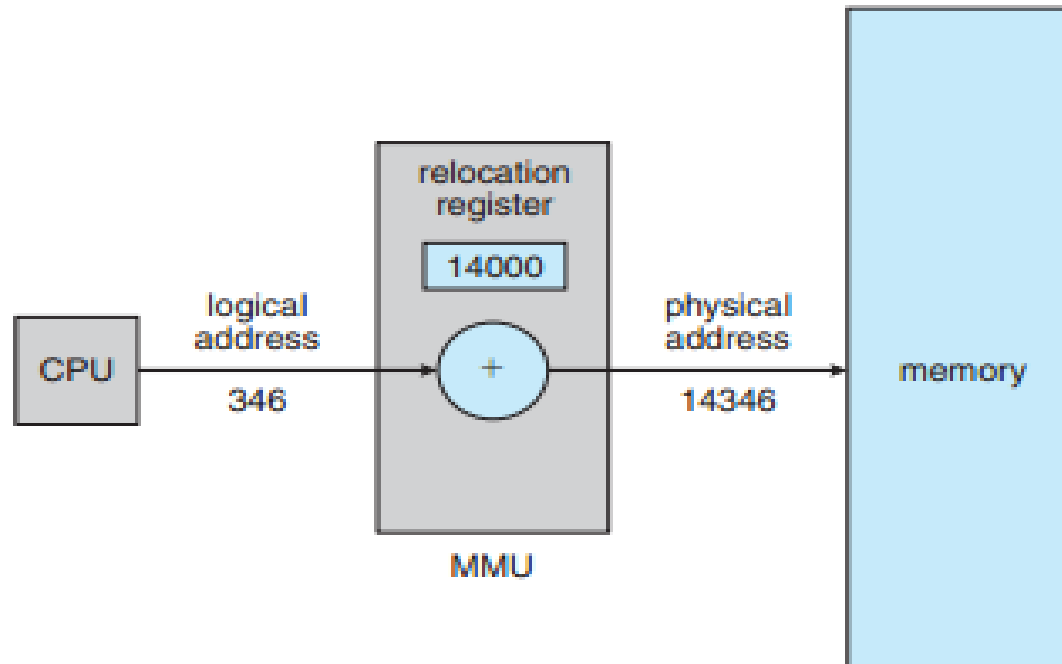


Figure 8.4 Dynamic relocation using a relocation register.

Logical Versus Physical Address Space(Contd.)

Parameter	LOGICAL ADDRESS	PHYSICAL ADDRESS
Basic	generated by CPU	location in a memory unit
Address Space	Logical Address Space is set of all logical addresses generated by CPU in reference to a program.	Physical Address is set of all physical addresses mapped to the corresponding logical addresses.
Visibility	User can view the logical address of a program.	User can never view physical address of program.
Generation	generated by the CPU	Computed by MMU
Access	The user can use the logical address to access the physical address.	The user can indirectly access physical address but not directly.



Swapping

- A process must be in memory to be executed. A process, however, can be swapped temporarily out of memory to a backing store and then brought back into memory for continued execution
- Standard swapping involves moving processes between main memory and a backing store.
- The backing store is commonly a fast disk. It must be large enough to accommodate copies of all memory images for all users, and it must provide direct access to these memory images.
- The system maintains a ready queue consisting of all processes whose memory images are on the backing store or in memory and are ready to run.
- Whenever the CPU scheduler decides to execute a process, it calls the dispatcher.

Swapping(Contd.)

- The dispatcher checks to see whether the next process in the queue is in memory. If it is not, and if there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the desired process. It then reloads registers and transfers control to the selected process.

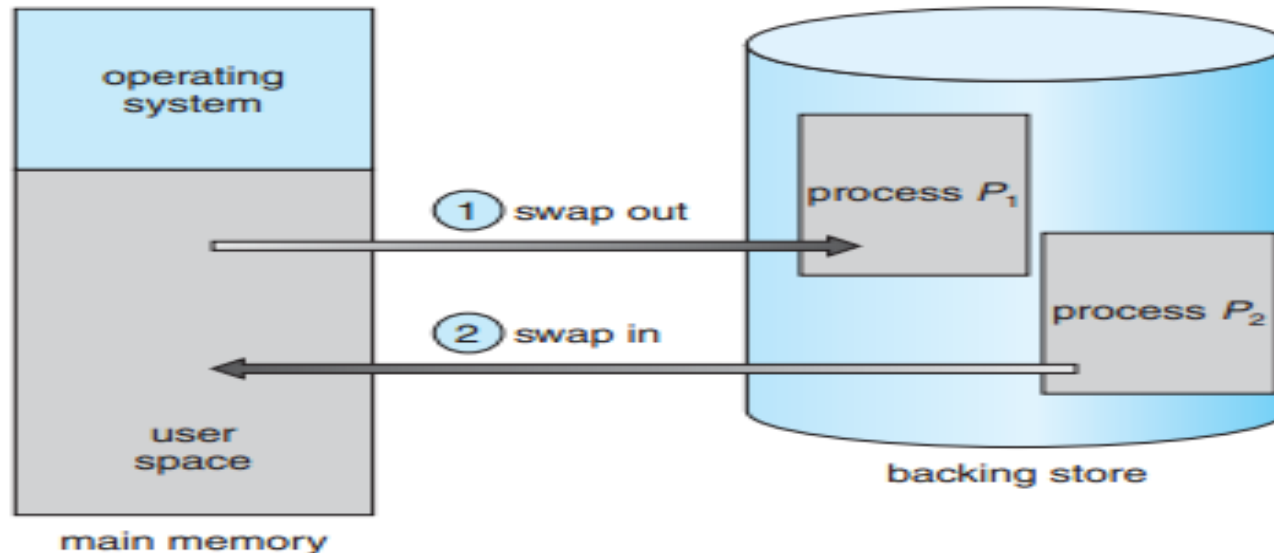


Figure 8.5 Swapping of two processes using a disk as a backing store.

Contiguous Memory Allocation

- One of the simplest methods for allocating memory is to divide memory into several **single/fixed-sized partitions**. Each partition may contain exactly one process. Thus, the degree of multiprogramming is bound by the number of partitions.
- In this multiple-partition method , when a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process.

Contiguous Memory Allocation(Contd.)

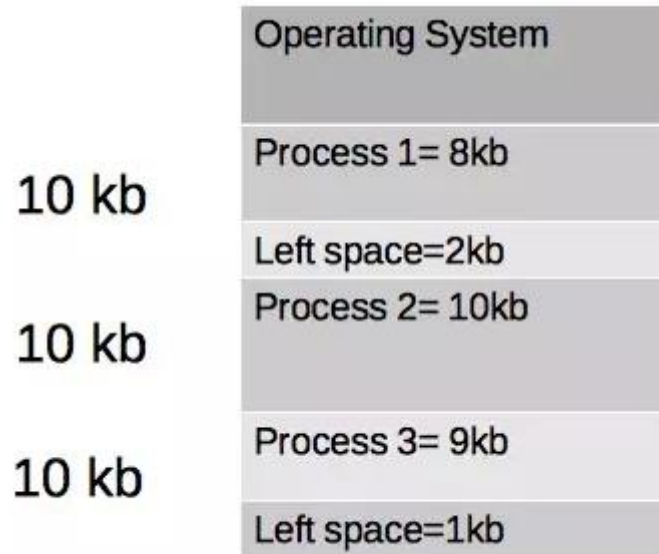


Fig. Single Partition/Fixed Size Partition

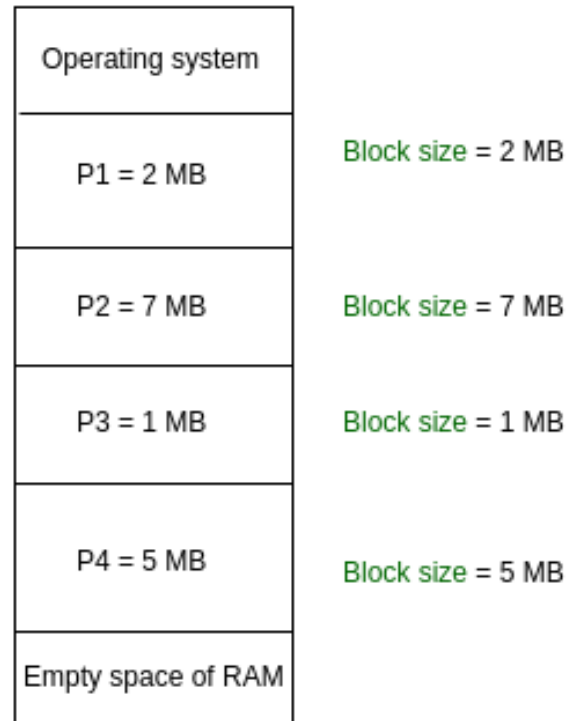
Contiguous Memory Allocation(Contd.)

Variable Partitioning – It is a part of Contiguous allocation technique. It is used to alleviate the problem faced by Fixed Partitioning. In contrast with fixed partitioning, partitions are not made before the execution or during system configure. Various **features** associated with variable Partitioning-

- Initially RAM is empty and partitions are made during the run-time according to process's need instead of partitioning during system configure.
- The size of partition will be equal to incoming process.
- The partition size varies according to the need of the process so that the internal fragmentation can be avoided to ensure efficient utilisation of RAM.
- Number of partitions in RAM is not fixed and depends on the number of incoming process and Main Memory's size.

Contiguous Memory Allocation(Contd.)

Dynamic partitioning



Partition size = process size
So, no internal Fragmentation



CONTIGUOUS/ DYNAMIC STORAGE ALLOCATION

- **First fit.** Allocate the first hole that is big enough. Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended. We can stop searching as soon as we find a free hole that is large enough.
- **Best fit.** Allocate the smallest hole that is big enough. We must search the entire list, unless the list is ordered by size. This strategy produces the smallest leftover hole.
- **Worst fit.** Allocate the largest hole. Again, we must search the entire list, unless it is sorted by size. This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.



Problem 01

- 1. Given five memory partitions of 100Kb, 500Kb, 200Kb, 300Kb, 600Kb (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of 212 Kb, 417 Kb, 112 Kb, and 426 Kb (in order)? Which algorithm makes the most efficient use of memory?



Solution 01

First-fit:

- 212K is put in 500K partition
- 417K is put in 600K partition
- 112K is put in 288K partition (new partition $288K = 500K - 212K$)
- 426K must wait

Best-fit:

- 212K is put in 300K partition
- 417K is put in 500K partition



Solution 01(Contd.)

- 112K is put in 200K partition
- 426K is put in 600K partition
- **Worst-fit:**
- 212K is put in 600K partition
- 417K is put in 500K partition
- 112K is put in 388K partition
- 426K must wait

- In this example, best-fit turns out to be the best.



Problem 02

Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB. These partitions need to be allocated to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order.

Perform the allocation of processes using-

- First Fit Algorithm
- Best Fit Algorithm
- Worst Fit Algorithm

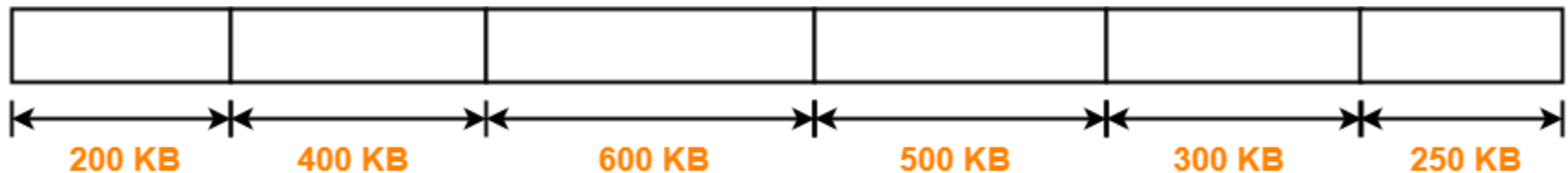
Problem 02

Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB. These partitions need to be allocated to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order.



Problem 02

Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB. These partitions need to be allocated to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order.



Main Memory

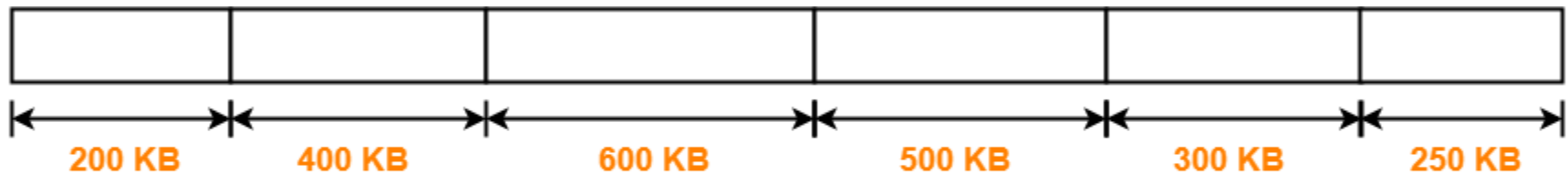
Allocation Using First Fit Algorithm-



Main Memory

Problem 02

Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB. These partitions need to be allocated to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order.



Main Memory

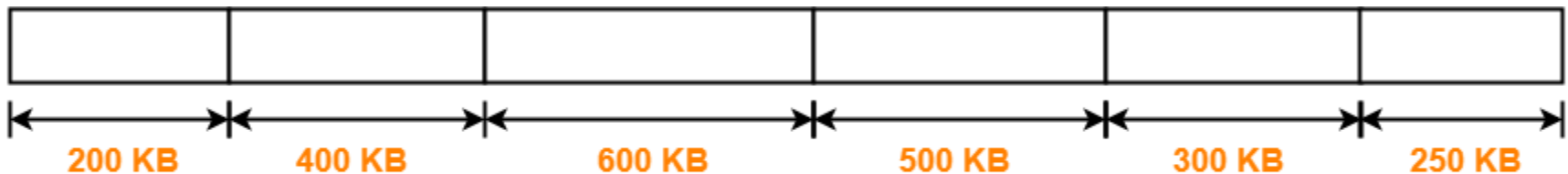
Allocation Using Best Fit Algorithm-



Main Memory

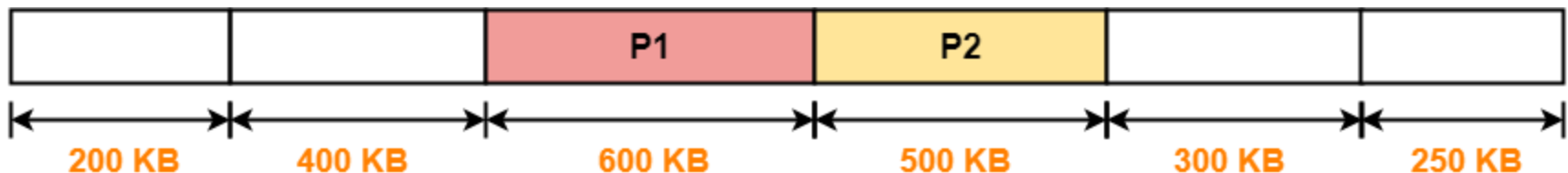
Problem 02

Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB. These partitions need to be allocated to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order.



Main Memory

Allocation Using Worst fit Algorithm-



Main Memory

Problem 03

- 9.6 Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358 KB, 200 KB, and 375 KB (in order)?

Answer:

- a. **First fit:**
- b. 115 KB is put in 300-KB partition, leaving 185 KB, 600 KB, 350 KB, 200 KB, 750 KB, 125 KB
- c. 500 KB is put in 600-KB partition, leaving 185 KB, 100 KB, 350 KB, 200 KB, 750 KB, 125 KB
- d. 358 KB is put in 750-KB partition, leaving 185 KB, 100 KB, 350 KB, 200 KB, 392 KB, 125 KB
- e. 200 KB is put in 350-KB partition, leaving 185 KB, 100 KB, 150 KB, 200 KB, 392 KB, 125 KB
- f. 375 KB is put in 392-KB partition, leaving 185 KB, 100 KB, 150 KB, 200 KB, 17 KB, 125 KB

Problem 03

Best Fit

- h. 115 KB is put in 125-KB partition, leaving 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, 10 KB
- i. 500 KB is put in 600-KB partition, leaving 300 KB, 100 KB, 350 KB, 200 KB, 750 KB, 10 KB
- j. 358 KB is put in 750-KB partition, leaving 300 KB, 100 KB, 350 KB, 200 KB, 392 KB, 10 KB
- k. 200 KB is put in 200-KB partition, leaving 300 KB, 100 KB, 350 KB, 0 KB, 392 KB, 10 KB
- l. 375 KB is put in 392-KB partition, leaving 300 KB, 100 KB, 350 KB, 0 KB, 17 KB, 10 KB

Problem 03

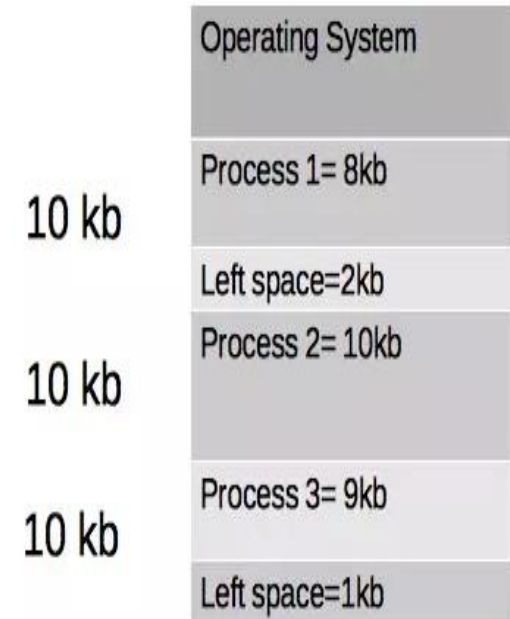
- m. **Worst fit:**
- n. 115 KB is put in 750-KB partition, leaving 300 KB, 600 KB, 350 KB, 200 KB, 635 KB, 125 KB
- o. 500 KB is put in 635-KB partition, leaving 300 KB, 600 KB, 350 KB, 200 KB, 135 KB, 125 KB
- p. 358 KB is put in 600-KB partition, leaving 300 KB, 242 KB, 350 KB, 200 KB, 135 KB, 125 KB
- q. 200 KB is put in 350-KB partition, leaving 300 KB, 242 KB, 150 KB, 200 KB, 135 KB, 125 KB
- r. 375 KB must wait

Contiguous v/s Non- Contiguous Memory Allocation

S.NO	Contiguous Memory Allocation	Non-Contiguous Memory Allocation
1.	Contiguous memory allocation allocates consecutive blocks of memory to a file/process.	Non-Contiguous memory allocation allocates separate blocks of memory to a file/process.
2.	Faster in Execution.	Slower in Execution.
3.	It is easier for the OS to control.	It is difficult for the OS to control.
4.	Overhead is minimum as not much address translations are there while executing a process.	More Overheads are there as there are more address translations.
5.	Both Internal fragmentation and external fragmentation occurs in Contiguous memory allocation method.	External fragmentation occurs in Non-Contiguous memory allocation method.
6.	It includes single partition allocation and multi-partition allocation.	It includes paging and segmentation.
7.	Wastage of memory is there.	No memory wastage is there.

Internal Fragmentation

- It arises when we use fixed sized partitioning.
- Some part of the memory is kept for operating system and the rest is available for user space.
- In this case the user space is divided into blocks of 10 KB each.
- When process 1 with size 8 KB is allocated a block of 10 KB, 2 KB space is left unused. When process 2 with size 10 KB is allocated a 10 KB block no space is left unused. When process 3 with size 9KB is allocated a 10 KB block, 1KB is left unused.
- Here 2 processes are allocated space more than required and this unused space is so small to store a new process and is wasted. This is called as INTERNAL FRAGMENTATION.

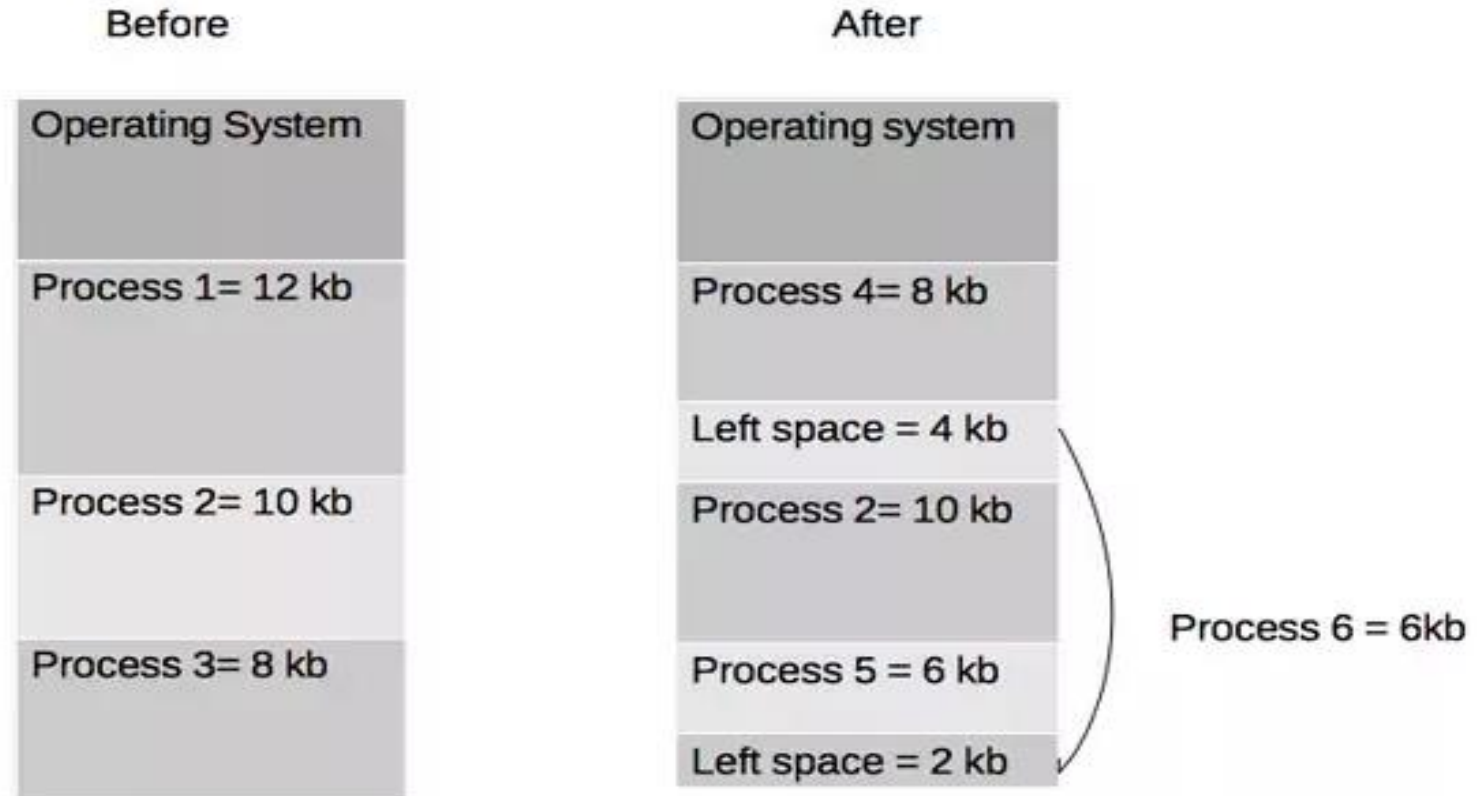




External Fragmentation

- It arises when dynamic partitioning technique is used.
- Here memory is allocated to the processes dynamically based on their size.
- So in the above example, the user space contains processes 1, 2 & 3. Out of which process 1 & 3 complete their execution and are swapped out and two other processes, process 4 & 5 are swapped in their places.
- Process 4 takes place of process 1 but as its size is only 8 KB, it is allocated only 8 KB and rest is left unused.
- Process 5 takes place of process 3. It is allocated 6 KB space and 8 KB is left unused.
- Now suppose a new process, process 6 wants to be swapped in and its size is 6 KB. Though we have total 6 KB space but we cannot service this request as these blocks are not contiguous(adjacent). This is called as **EXTERNAL FRAGMENTATION**.

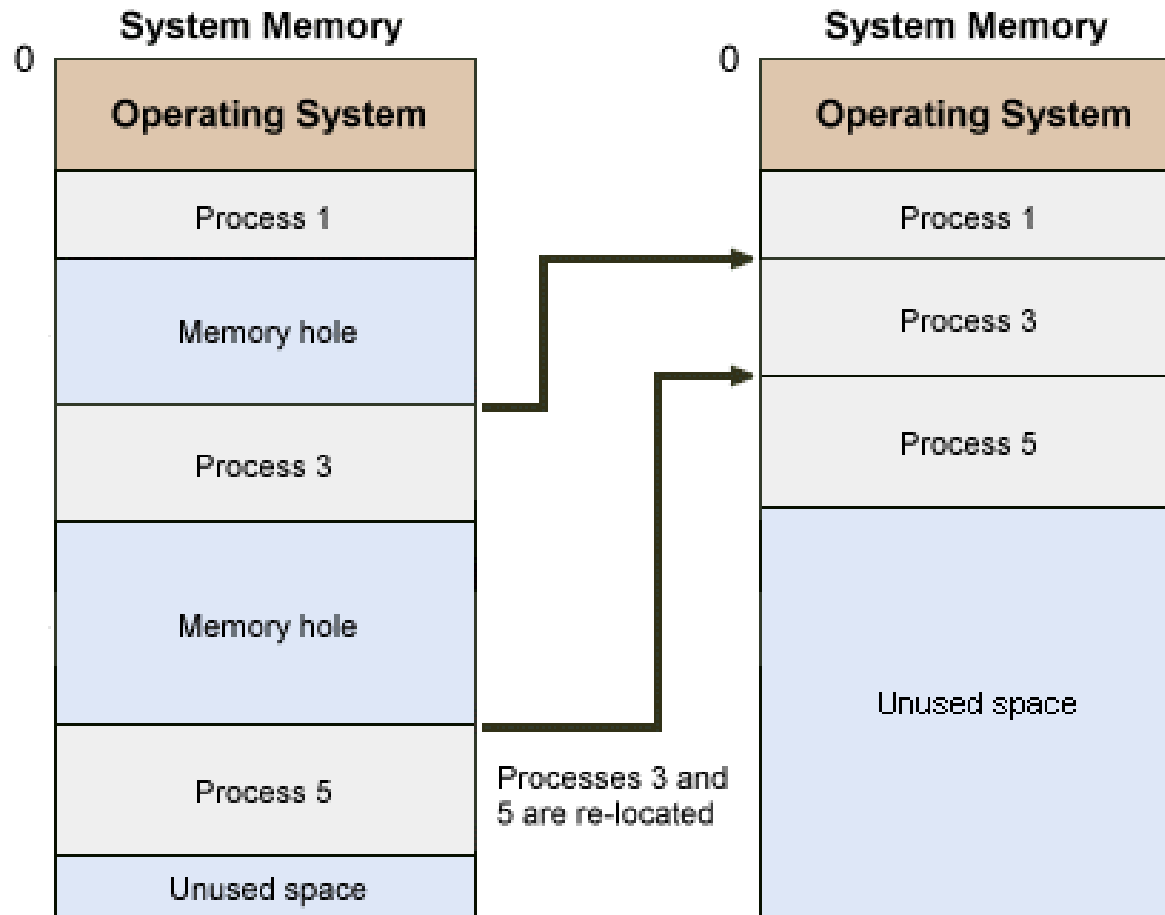
External Fragmentation



Compaction

- We got to know that the dynamic partitioning suffers from external fragmentation. However, this can cause some serious problems.
- To avoid compaction, we need to change the rule which says that the process can't be stored in the different places in the memory.
- We can also use compaction to minimize the probability of external fragmentation. In compaction, all the free partitions are made contiguous and all the loaded partitions are brought together.
- By applying this technique, we can store the bigger processes in the memory. The free partitions are merged which can now be allocated according to the needs of new processes. This technique is also called defragmentation.

Compaction



Comparison Between External & Internal Fragmentation

BASIS FOR COMPARISON	INTERNAL FRAGMENTATION	EXTERNAL FRAGMENTATION
Basic	It occurs when fixed sized memory blocks are allocated to the processes.	It occurs when variable size memory space are allocated to the processes dynamically.
Occurrence	When the memory assigned to the process is slightly larger than the memory requested by the process this creates free space in the allocated block causing internal fragmentation.	When the process is removed from the memory, it creates the free space in the memory causing external fragmentation.
Solution	The memory must be partitioned into variable sized blocks and assign the best fit block to the process.	Compaction, paging and segmentation.

Paging

- **Paging and Segmentation** permits the physical address space of a process to be non-contiguous.
- Paging is another memory-management scheme that offers this advantage.
- However, paging avoids external fragmentation and the need for compaction, whereas segmentation does not.
- It also solves the considerable problem of fitting memory chunks of varying sizes onto the backing store.



Basic Method: Paging

- The basic method for implementing paging involves breaking physical memory into fixed-sized blocks called frames and breaking logical memory into blocks of the same size called pages.
- When a process is to be executed, its pages are loaded into any available memory frames from their source (a file system or the backing store).
- The backing store is divided into fixed-sized blocks that are the same size as the memory frames or clusters of multiple frames
- Every address generated by the CPU is divided into two parts: a page number (p) and a page offset (d).
- The page number is used as an index into a page table. The page table contains the base address of each page in physical memory.
- This base address is combined with the page offset to define the physical memory address that is sent to the memory unit.

Paging Hardware

Address generated by CPU is divided into

- **Page number(p):** Number of bits required to represent the pages in Logical Address Space or Page number
- **Page offset(d):** Number of bits required to represent particular word in a page or page size of Logical Address Space or word number of a page or page offset.

Physical Address is divided into

- **Frame number(f):** Number of bits required to represent the frame of Physical Address Space or Frame number.
- **Frame offset(d):** Number of bits required to represent particular word in a frame or frame size of Physical Address Space or word number of a frame or frame offset.

Paging Hardware

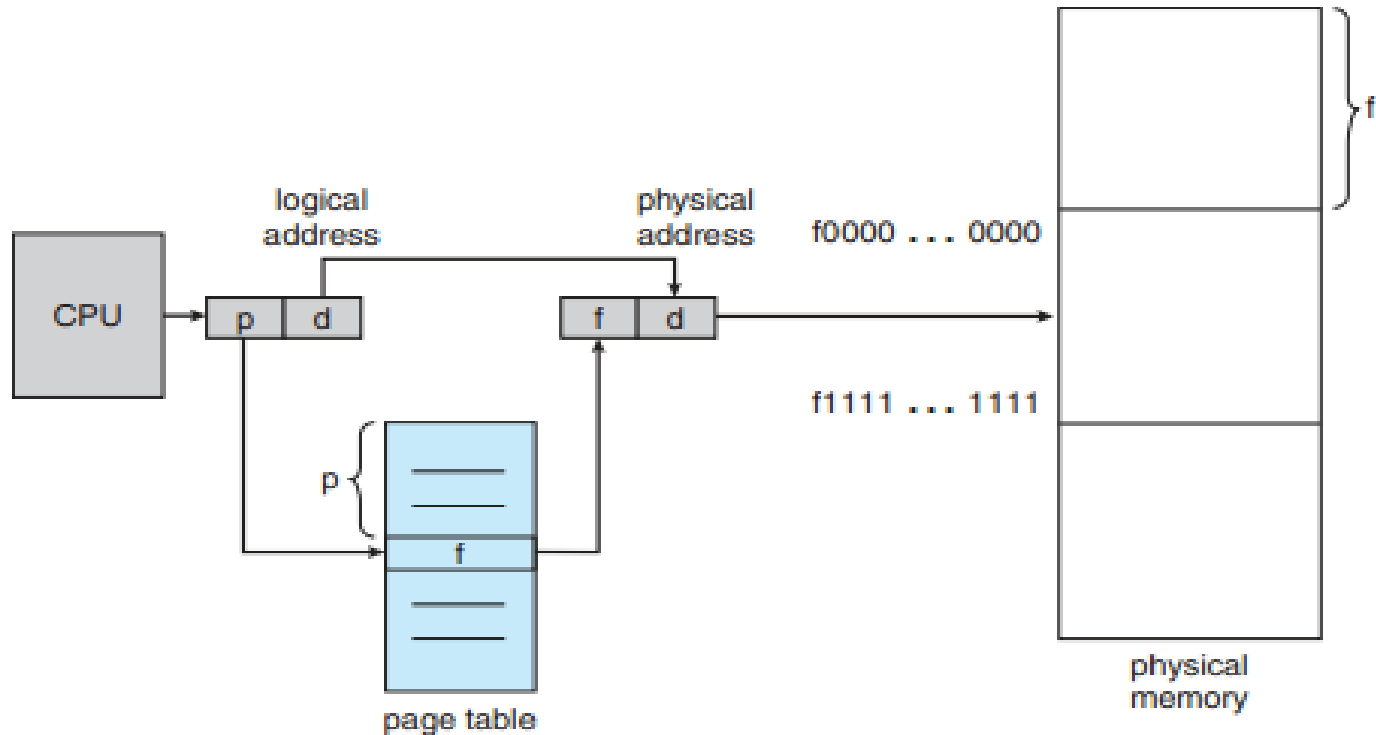


Figure 8.10 Paging hardware.

Paging

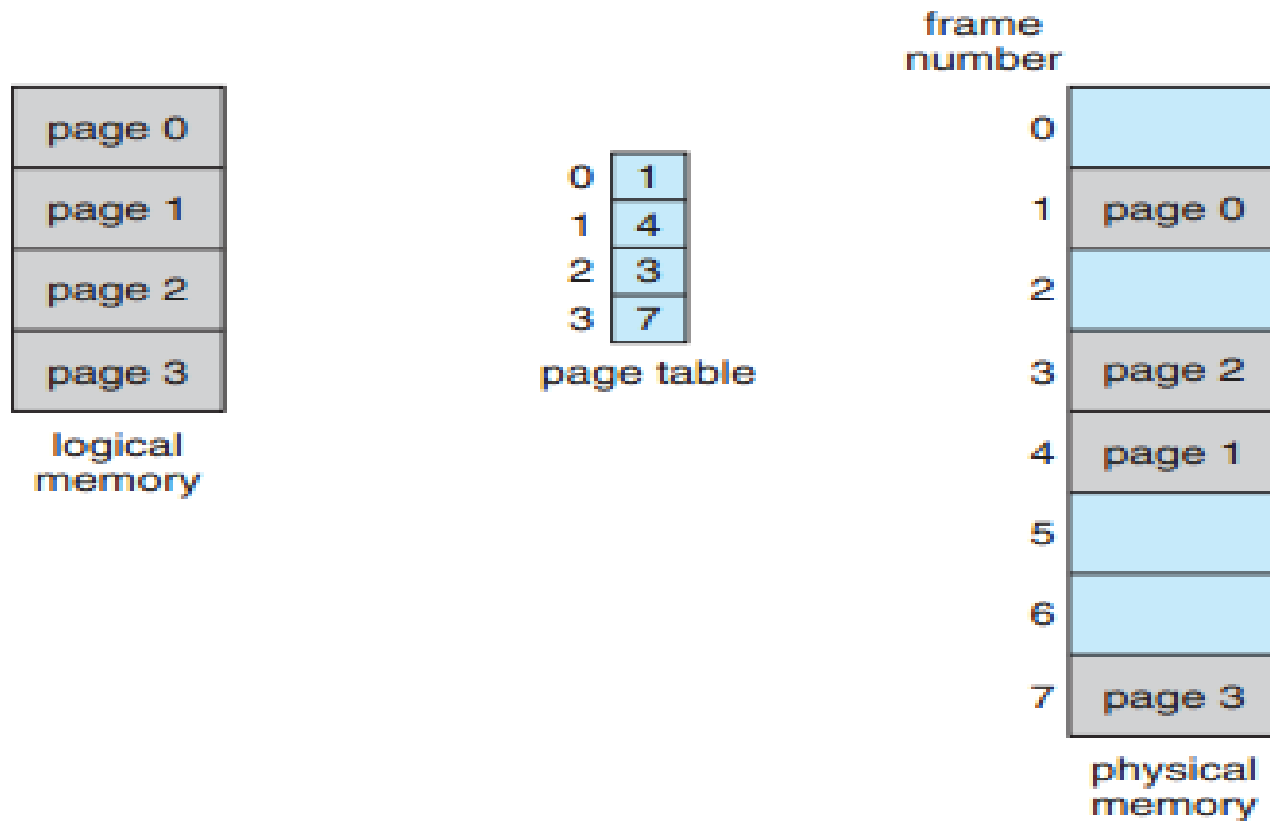
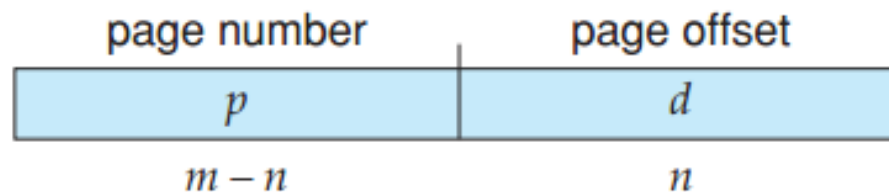


Figure 8.11 Paging model of logical and physical memory.

Paging

The page size (like the frame size) is defined by the hardware. The size of a page is a power of 2, varying between 512 bytes and 1 GB per page, depending on the computer architecture. The selection of a power of 2 as a page size makes the translation of a logical address into a page number and page offset particularly easy. If the size of the logical address space is 2^m , and a page size is 2^n bytes, then the high-order $m - n$ bits of a logical address designate the page number, and the n low-order bits designate the page offset. Thus, the logical address is as follows:



Paging

As a concrete (although minuscule) example, consider the memory in Figure 8.12. Here, in the logical address, $n = 2$ and $m = 4$. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages), we show how the programmer's view of memory can be mapped into physical memory. Logical address 0 is page 0, offset 0. Indexing into the page table, we find that page 0

is in frame 5. Thus, logical address 0 maps to physical address 20 $[= (5 \times 4) + 0]$. Logical address 3 (page 0, offset 3) maps to physical address 23 $[= (5 \times 4) + 3]$. Logical address 4 is page 1, offset 0; according to the page table, page 1 is mapped to frame 6. Thus, logical address 4 maps to physical address 24 $[= (6 \times 4) + 0]$. Logical address 13 maps to physical address 9.

Paging

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Paging example for a 32-byte memory with 4-byte pages.

Points to remember

For Main Memory-

- Physical Address Space = Size of main memory
- Size of main memory = Total number of frames x Page size
- Frame size = Page size
- If number of frames in main memory = 2^X , then number of bits in frame number = X bits
- If Page size = 2^X Bytes, then number of bits in page offset = X bits
- If size of main memory = 2^X Bytes, then number of bits in physical address = X bits

Points to remember

- 1 byte = 8 bits
- 1 kilobyte (K / Kb) = 2^{10} bytes = 1,024 bytes
- 1 megabyte (M / MB) = 2^{20} bytes = 1,048,576 bytes
- 1 gigabyte (G / GB) = 2^{30} bytes = 1,073,741,824 bytes
- 1 terabyte (T / TB) = 2^{40} bytes = 1,099,511,627,776 bytes
- 1 petabyte (P / PB) = 2^{50} bytes = 1,125,899,906,842,624 bytes
- 1 exabyte (E / EB) = 2^{60} bytes = 1,152,921,504,606,846,976 bytes



Problems on Paging

Q1) Calculate the size of memory if its address consists of 22 bits and the memory is 2-byte addressable.

Ans. We have-

Number of locations possible with 22 bits = 2^{22} locations

It is given that the size of one location = 2 bytes

Size of main memory = Total number of frames x Page size

Thus, Size of memory

$$= 2^{22} \times 2$$

$$= 2^{23} \text{ bytes}$$

$$= 8 \text{ MB}$$



Problems on Paging

Q:2: Calculate the number of bits required in the address for memory having size of 16 GB. Assume the memory is 4-byte addressable.

Problems on Paging

Q:2: Calculate the number of bits required in the address for memory having size of 16 GB. Assume the memory is 4-byte addressable.

Solution-

Let 'n' number of bits are required.

Size of main memory = Total number of frames x Page size

Then, Size of memory = $2^n \times 4$ bytes.

Since, the given memory has size of 16 GB, so we have-

$$2^n \times 4 \text{ bytes} = 16 \text{ GB}$$

$$2^n \times 4 = 16 \text{ G}$$

$$2^n \times 2^2 = 2^{34}$$

$$2^n = 2^{32}$$

$$\therefore n = 32 \text{ bits}$$



Points to remember

For Process-

- Virtual Address Space = Size of process
- Number of pages the process is divided = $\text{Process size} / \text{Page size}$
- If process size = 2^X bytes, then number of bits in virtual address space = X bits



Problems on Paging

Q1) Evaluating the maximum number of pages needed If a system supports 16 bit address line and 1K page size.

Ans. Virtual Address space= 2^{16} bytes

Page size= 1k = 2^{10} bytes

Number of pages= Virtual Address space/page size

$$= 2^{16}/2^{10}$$

$$= 2^6 = 64 \text{ pages}$$

Points to remember

For Page Table-

- Size of page table = Number of entries in page table x Page table entry size
- Number of entries in pages table = Number of pages the process is divided
- Page table entry size = Number of bits in frame number + Number of bits used for optional fields if any

Points to remember

NOTE-

- In general, if the given address consists of 'n' bits, then using 'n' bits, 2^n locations are possible.
- Then, size of memory = $2^n \times$ Size of one location.
- If the memory is byte-addressable, then size of one location = 1 byte.
- Thus, size of memory = 2^n bytes.
- If the memory is word-addressable where 1 word = m bytes, then size of one location = m bytes.
- Thus, size of memory = $2^n \times m$ bytes.
- Let's say, Page size = 4 KB = 2^{12} Bytes then Page offset = 12



Problems on Paging

Q4) Consider a system with byte-addressable memory, 32 bit logical addresses, 4 kilobyte page size and page table entries of 4 bytes each. The size of the page table in the system in megabytes is ____.

Ans. Given-Number of bits in logical address = 32 bits

Page size = 4KB

Page table entry size = 4 bytes

Process Size-

Number of bits in logical address = 32 bits

Thus,

Process size

= 2^{32} B

= 4 GB



Problems on Paging(contd.)

Number of Entries in Page Table-

Number of pages the process is divided

= Process size / Page size

= 4 GB / 4 KB

= 2^{20} pages

Thus, Number of entries in page table = 2^{20} entries

Page Table Size-

Page table size

= Number of entries in page table x Page table entry size

= 2^{20} x 4 bytes

= 4 MB



Problems on Paging

Q: Consider a machine with 64 MB physical memory and a 32 bit virtual address space. If the page size is 4 KB, what is the approximate size of the page table?

16 MB

8 MB

2 MB

24 MB



Problems on Paging

Q: Consider a machine with 64 MB physical memory and a 32 bit virtual address space. If the page size is 4 KB, what is the approximate size of the page table?

Ans

Given-

Size of main memory = 64 MB

Number of bits in virtual address space = 32 bits

Page size = 4 KB

We will consider that the memory is byte addressable.

Number of Bits in Physical Address-

Size of main memory

= 64 MB

= 2^{26} B

Thus, Number of bits in physical address = 26 bits



Problems on Paging(contd.)

Number of Frames in Main Memory-

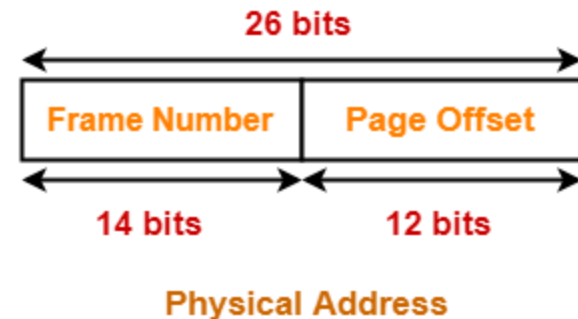
Number of frames in main memory
= Size of main memory / Frame size
= 64 MB / 4 KB
= $2^{26} \text{ B} / 2^{12} \text{ B}$
= 2^{14}

Thus, Number of bits in frame number = 14 bits

Number of Bits in Page Offset-

We have,
Page size
= 4 KB
= 2^{12} B

Thus, Number of bits in page offset = 12 bits





Problems on Paging(contd.)

Process Size-

Number of bits in virtual address space = 32 bits

Thus,

Process size

$$= 2^{32} \text{ B}$$

$$= 4 \text{ GB}$$

Number of Entries in Page Table-

Number of pages the process is divided

$$= \text{Process size} / \text{Page size}$$

$$= 4 \text{ GB} / 4 \text{ KB}$$

$$= 2^{20} \text{ pages}$$

Thus, Number of entries in page table = 2^{20} entries



Problems on Paging(contd.)

Page Table Size-

Page table size

= Number of entries in page table x Page table entry size

= Number of entries in page table x Number of bits in frame number

= $2^{20} \times 14$ bits

= $2^{20} \times 16$ bits (Approximating 14 bits $\approx$ 16 bits)

= $2^{20} \times 2$ bytes

= 2 MB

Thus, Option (C) is correct.



Problems on Paging

Q: Consider a logical space of 64 pages of 1024 words each mapped onto a physical memory of 32 frames.

- a. how many bits are there in the logical address
- b. how many bits are there in physical address

Ans.

The logical address requires 6 bits for the page number because there are $64 = 2^6$ of them. Then, the logical address requires 10 bits of the offset because there are $1024 = 2^{10}$ of them. So, the logical address requires a total is 16 bits.

The physical address requires 5 bits for the frame number because there are $32 = 2^5$ of them. Then, the physical address also requires 10 bits of the offset because there are $1024 = 2^{10}$ of them. So, the physical address requires a total is 15 bits.

Disadvantage Of Paging

One major disadvantage of paging is-

- It increases the **effective access time** due to increased number of memory accesses.
- One memory access is required to get the frame number from the page table.
- Another memory access is required to get the word from the page.

Paging

Why are page sizes always powers of 2?

Answer:

Recall that paging is implemented by breaking up an address into a page and offset number. It is most efficient to break the address into X page bits and Y offset bits, rather than perform arithmetic on the address to calculate the page number and offset. Because each bit position represents a power of 2, splitting an address between bits results in a page size that is a power of 2.

References

- “Operating System Concepts” by Avi Silberschatz and Peter Galvin
- “Operating Systems: Internals and Design Principles” by William Stallings
- www.tutorialpoint.com
- www.javapoint.com



Thank You

For any Query, you can contact

Er. Inderjeet Singh(e8822)

Ph. No: 86-99-100-160

Email id: inderjeet.e8822@cumail.in



University Institute of Engineering

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Unit- 2

Subject Name : Operating System

Chapter : Memory Management- (Part-2)

Prepared By: Er. Inderjeet Singh(e8822)



Outline

- Paging with TLB
- Exercise on Paging with TLB
- Segmentation and Exercise Problems

Paging with TLB:

- Translation Look-Aside Buffer, a table in the processors memory that contains information about the pages in memory the processor has accessed recently. The TLB enables faster computing because it allows the address processing to take place independent of the normal address-translation pipeline.
- The TLB is associative, high-speed memory. Each entry in the TLB consists of two parts: a key (or tag) and a value. When the associative memory is presented with an item, the item is compared with all keys simultaneously. If the item is found, the corresponding value field is returned. The search is fast; a TLB lookup in modern hardware is part of the instruction pipeline, essentially adding no performance penalty. To be able to execute the search within a pipeline step, however, the TLB must be kept small. It is typically between 32 and 1,024 entries in size.

Paging with TLB:

The logical address generated by the CPU is translated into the physical address using following steps-

Step-01:

CPU generates a logical address consisting of two parts-

- Page Number
- Page Offset

Step-02:

- TLB is checked to see if it contains an entry for the referenced page number.
- The referenced page number is compared with the TLB entries all at once.



Paging with TLB:

Now, two cases are possible-

Case-01: If there is a TLB hit-

- If TLB contains an entry for the referenced page number, a TLB hit occurs.
- In this case, TLB entry is used to get the corresponding frame number for the referenced page number.

Case-02: If there is a TLB miss-

- If TLB does not contain an entry for the referenced page number, a TLB miss occurs.
- In this case, page table is used to get the corresponding frame number for the referenced page number.
- Then, TLB is updated with the page number and frame number for future references.

Paging with TLB:

Step-03:

- After the frame number is obtained, it is combined with the page offset to generate the physical address.
- Then, physical address is used to read the required word from the main memory.

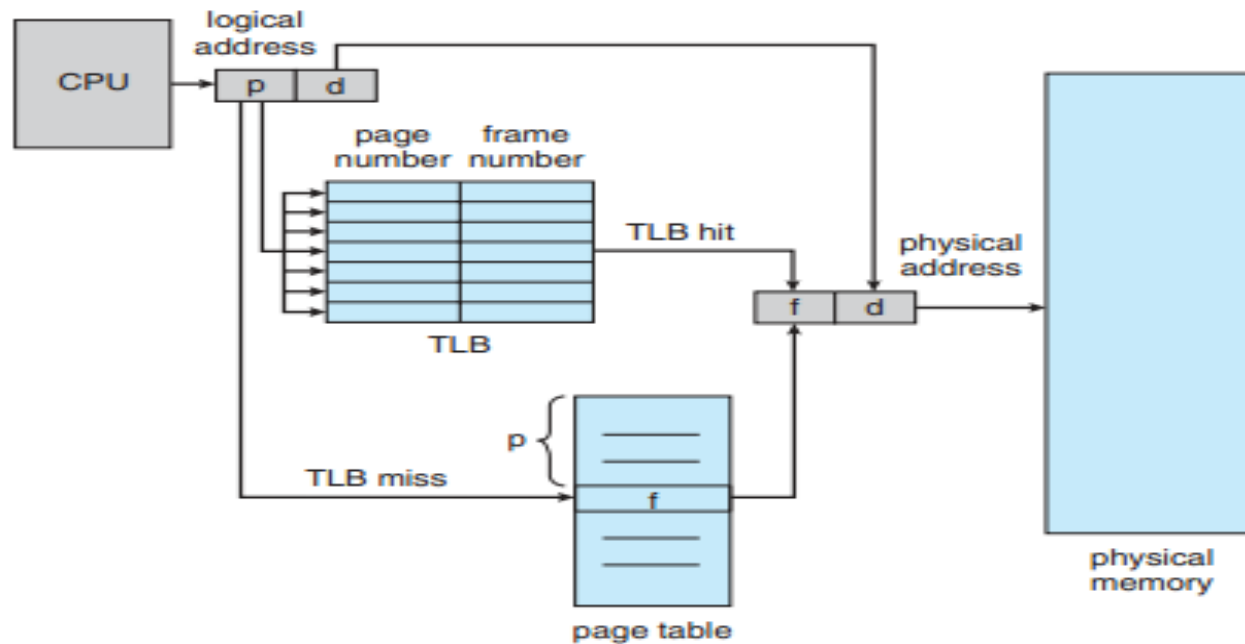


Figure 8.14 Paging hardware with TLB.

Paging with TLB:

- The TLB is used with page tables in the following way. The TLB contains only a few of the page-table entries. When a logical address is generated by the CPU, its page number is presented to the TLB. If the page number is found, its frame number is immediately available and is used to access memory.
- In addition, we add the page number and frame number to the TLB, so that they will be found quickly on the next reference. If the TLB is already full of entries, an existing entry must be selected for replacement.

Paging with TLB:

The **advantages of using TLB** are-

- TLB reduces the effective access time.
- Only one memory access is required when TL

Disadvantages- A major disadvantage of using TLB is-

- After some time of running the process, when TLB hits increases and process starts to run smoothly, a context switching occurs.
- The entire content of the TLB is flushed.
- Then, TLB is again updated with the currently running process. This happens again and again.

Effective Access Time

- In a single level paging using TLB, the effective access time is given as-

$$\begin{aligned} \text{Effective Access Time} = & \\ & \text{Hit ratio of TLB} \times \{ \text{Access time of TLB} + \text{Access time of main memory} \} \\ & + \\ & \text{Miss ratio of TLB} \times \{ \text{Access time of TLB} + 2 \times \text{Access time of main memory} \} \end{aligned}$$



Problems on Paging with TLB

Problem 1: A paging scheme uses a Translation Lookaside buffer (TLB). A TLB access takes 10 ns and a main memory access takes 50 ns. What is the effective access time (in ns) if the TLB hit ratio is 90% and there is no page fault?

- a) 54
- b) 60
- c) 65
- d) 75

Solution: Given-

TLB access time = 10 ns

Main memory access time = 50 ns

TLB Hit ratio = 90% = 0.9

Problems on Paging with TLB

Calculating TLB Miss Ratio-

$$\begin{aligned}\text{TLB Miss ratio} &= 1 - \text{TLB Hit ratio} \\ &= 1 - 0.9 \\ &= 0.1\end{aligned}$$

Calculating Effective Access Time-

Substituting values in the above formula, we get-

Effective Access Time

$$\begin{aligned}&= 0.9 \times \{ 10 \text{ ns} + 50 \text{ ns} \} + 0.1 \times \{ 10 \text{ ns} + 2 \times 50 \text{ ns} \} \\ &= 0.9 \times 60 \text{ ns} + 0.1 \times 110 \text{ ns} \\ &= 54 \text{ ns} + 11 \text{ ns} \\ &= 65 \text{ ns}\end{aligned}$$

Thus, Option (C) is correct.



Problems on Paging with TLB

Problem-02: A paging scheme uses a Translation Lookaside buffer (TLB). The effective memory access takes 160 ns and a main memory access takes 100 ns. What is the TLB access time (in ns) if the TLB hit ratio is 60% and there is no page fault?

- a) 54
- b) 60
- c) 20
- d) 75



Problems on Paging with TLB

Given-

Effective access time = 160 ns

Main memory access time = 100 ns

TLB Hit ratio = 60% = 0.6

Calculating TLB Miss Ratio-

TLB Miss ratio

= 1 – TLB Hit ratio

= 1 – 0.6

= 0.4

Calculating TLB Access Time-

Let TLB access time = T ns.

Substituting values in the above formula, we get-

$160 \text{ ns} = 0.6 \times \{ T + 100 \text{ ns} \} + 0.4 \times \{ T + 2 \times 100 \text{ ns} \}$

$160 = 0.6 \times T + 60 + 0.4 \times T + 80$

$160 = T + 140$

$T = 160 - 140$

$T = 20$

Segmentation

- Segmentation is a memory-management scheme that supports this programmer view of memory.
- A logical address space is a collection of segments. Each segment has a name and a length.
- The addresses specify both the segment name and the offset within the segment.
- The programmer therefore specifies each address by two quantities: a segment name and an offset.
- For simplicity of implementation, segments are numbered and are referred to by a segment number, rather than by a segment name.
- Thus, a logical address consists of a two tuple: <segment-number, offset>

Segmentation Hardware

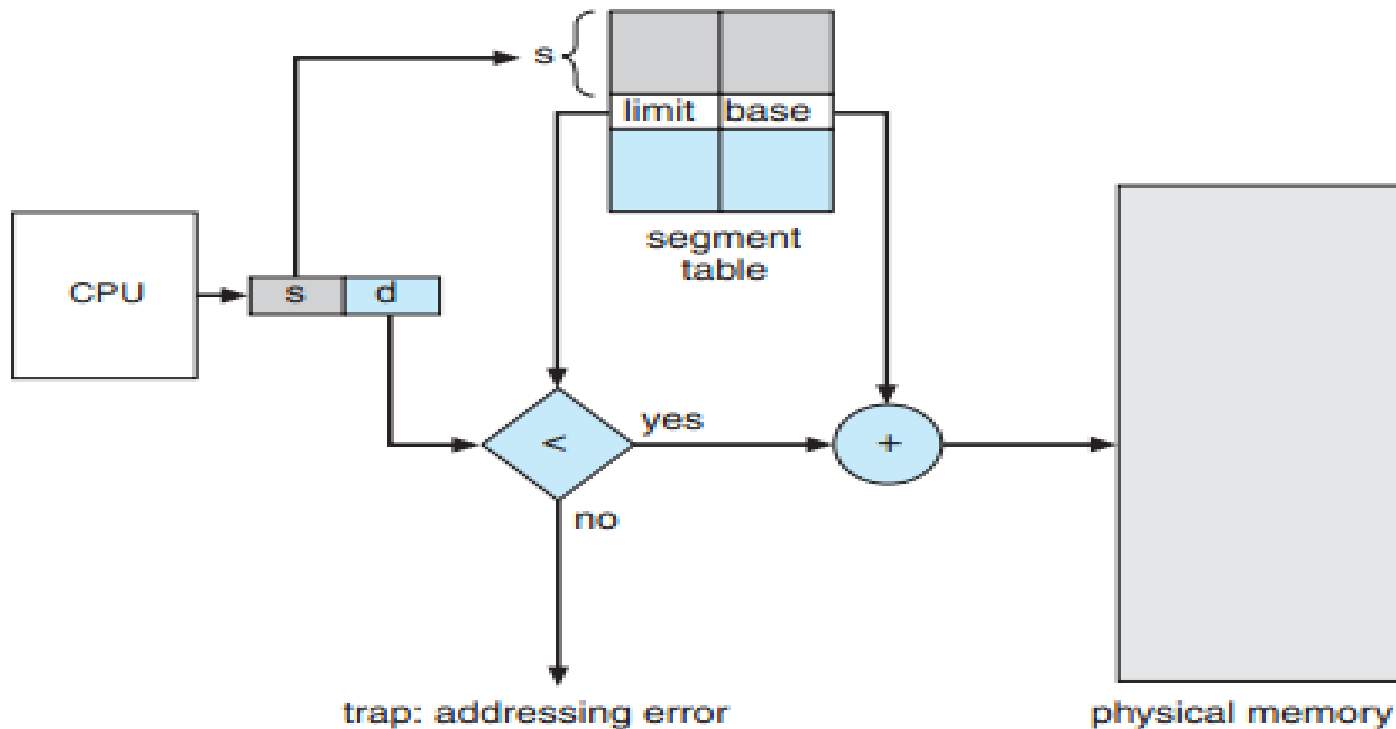


Figure 8.8 Segmentation hardware.

Segmentation(Contd.)

- Each entry in the segment table has a segment base and a segment limit.
- The segment base contains the starting physical address where the segment resides in memory, and the segment limit specifies the length of the segment.
- The use of a segment table is illustrated in Figure 8.8.
- A logical address consists of two parts: a segment number, s , and an offset into that segment, d .
- The segment number is used as an index to the segment table.
- The offset d of the logical address must be between 0 and the segment limit.
- If it is not, we trap to the operating system (logical addressing attempt beyond end of segment).

Segmentation (Contd.)

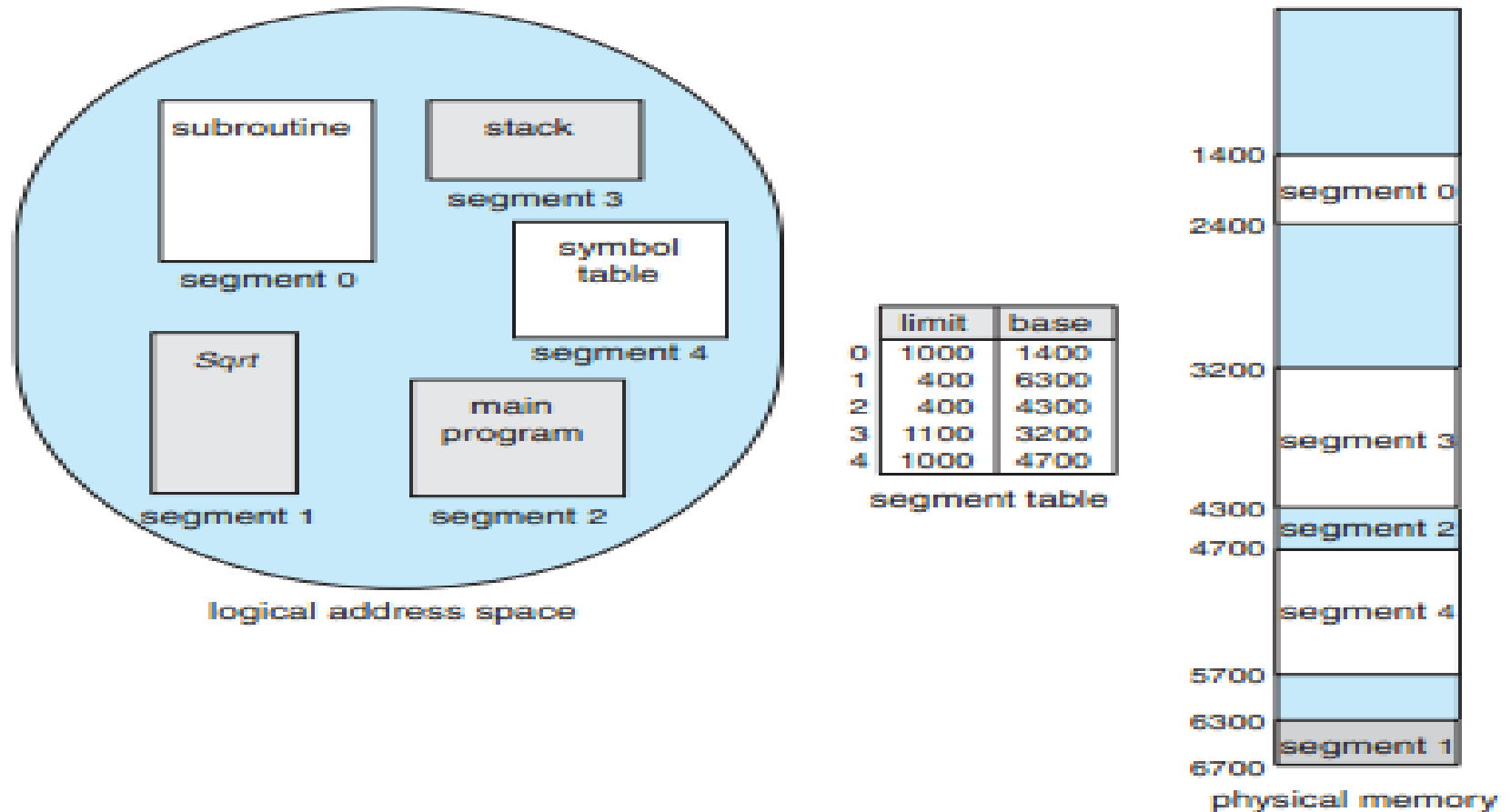


Figure 8.9 Example of segmentation.

Segmentation (Contd.)

- Consider the situation shown in Figure 8.9. We have five segments numbered from 0 through 4. The segments are stored in physical memory as shown.
- The segment table has a separate entry for each segment, giving the beginning address of the segment in physical memory (or base) and the length of that segment (or limit).
- For example, segment 2 is 400 bytes long and begins at location 4300. Thus, a reference to byte 53 of segment 2 is mapped onto location $4300 + 53 = 4353$.
- A reference to segment 3, byte 852, is mapped to 3200 (the base of segment 3) + 852 = 4052.
- A reference to byte 1222 of segment 0 would result in a trap to the operating system, as this segment is only 1,000 bytes long.

Advantages of SEGMENTATION

- In the Segmentation technique, the segment table is mainly used to keep the record of segments. Also, the segment table occupies less space as compared to the paging table.
- There is no Internal Fragmentation.
- Segmentation generally allows us to divide the program into modules that provide better visualization.
- Segments are of variable size.

Disadvantages of SEGMENTATION

- Maintaining a segment table for each process leads to overhead
- This technique is expensive.
- The time is taken in order to fetch the instruction increases since now two memory accesses are required.
- Segments are of unequal size in segmentation and thus are not suitable for swapping.
- This technique leads to external fragmentation as the free space gets broken down into smaller pieces along with the processes being loaded and removed from the main memory then this will result in a lot of memory waste.

S.NO	Paging	Segmentation
1.	In paging, program is divided into fixed or mounted size pages.	In segmentation, program is divided into variable size sections.
2.	For paging operating system is accountable.	For segmentation compiler is accountable.
3.	Page size is determined by hardware.	Here, the section size is given by the user.
4.	Paging could result in internal fragmentation.	Segmentation could result in external fragmentation.
5.	In paging, logical address is split into page number and page offset.	Here, logical address is split into section number and section offset.
6.	Paging comprises a page table which encloses the base address of every page.	While segmentation also comprises the segment table which encloses segment number and segment offset.
7.	Page table is employed to keep up the page data.	Section Table maintains the section data.
8.	In paging, operating system must maintain a free frame list.	In segmentation, operating system maintain a list of holes in main memory.
9.	Paging is invisible to the user.	Segmentation is visible to the user.
10.	In paging, processor needs page number, offset to calculate absolute address.	In segmentation, processor uses segment number, offset to calculate full address.

Exercise on SEGMENTATION

8.28 Consider the following segment table:

<u>Segment</u>	<u>Base</u>	<u>Length</u>
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are the physical addresses for the following logical addresses?

- a. 0,430
- b. 1,10
- c. 2,500
- d. 3,400
- e. 4,112



Exercise on SEGMENTATION

Problem- Consider the following segment table-

Segment No.	Base	Limit
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

Which of the following logical address will produce trap addressing error?

- 0, 430
- 1, 10
- 2, 500
- 3, 400
- 4, 112

Exercise on SEGMENTATION

Solution-

In a segmentation scheme, the generated logical address consists of two parts-

Segment Number

Segment Offset

We know-

Segment Offset must always lie in the range $[0, \text{limit}-1]$.

If segment offset becomes greater than or equal to the limit of segment, then trap addressing error is produced.



Exercise on SEGMENTATION

Solution-

Option-A: 0, 430-

Here,

Segment Number = 0

Segment Offset = 430

We have,

In the segment table, limit of segment-0 is 600.

Thus, segment offset must always lie in the range = $[0, 600-1] = [0, 599]$

Now,

Since generated segment offset lies in the above range, so request generated is valid.

Therefore, no trap will be produced.

Physical Address = $219 + 430 = 649$



Exercise on SEGMENTATION

Solution-

Option-B: 1, 10-

Here,

Segment Number = 1

Segment Offset = 10

We have,

In the segment table, limit of segment-1 is 14.

Thus, segment offset must always lie in the range = $[0, 14-1] = [0, 13]$

Now,

Since generated segment offset lies in the above range, so request generated is valid.

Therefore, no trap will be produced.

Physical Address = $2300 + 10 = 2310$



Exercise on SEGMENTATION

Solution-

Option-C: 2, 500-

Here,

Segment Number = 2

Segment Offset = 500

We have,

In the segment table, limit of segment-2 is 100.

Thus, segment offset must always lie in the range = $[0, 100-1] = [0, 99]$

Now,

Since generated segment offset does not lie in the above range, so request generated is invalid.

Therefore, trap will be produced.



Exercise on SEGMENTATION

Solution-

Option-D: 3, 400-

Here,

Segment Number = 3

Segment Offset = 400

We have,

In the segment table, limit of segment-3 is 580.

Thus, segment offset must always lie in the range = $[0, 580-1] = [0, 579]$

Now,

Since generated segment offset lies in the above range, so request generated is valid.

Therefore, no trap will be produced.

Physical Address = $1327 + 400 = 1727$



Exercise on SEGMENTATION

Solution-

Option-E: 4, 112-

Here,

Segment Number = 4

Segment Offset = 112

We have,

In the segment table, limit of segment-4 is 96.

Thus, segment offset must always lie in the range = $[0, 96-1] = [0, 95]$

Now,

Since generated segment offset doesn't lie in the above range, so request generated is not valid.

Therefore, trap will be produced.



PAGING with SEGMENTATION

Major Limitation of Single Level Paging

- A big challenge with single level paging is that if the logical address space is large, then the page table may take up a lot of space in main memory.
- For instance, consider that logical address is 32 bit and each page is 4 KB, the number of pages will be 2^{20} pages. The page table without additional bits will be of the size $20 \text{ bits} * 2^{20}$ or 2.5 MB.
- Since each process has its own page table, a lot of memory will be consumed when single level paging is used.
- A solution to the problem is to use segmentation along with paging to reduce the size of page table.

PAGING with SEGMENTATION

- Process is first divided into segments and then each segment is divided into pages.
- These pages are then stored in the frames of main memory.
- A page table exists for each segment that keeps track of the frames storing the pages of that segment.
- Each page table occupies one frame in the main memory.
- Number of entries in the page table of a segment = Number of pages that segment is divided.
- A segment table exists that keeps track of the frames storing the page tables of segments.
- Number of entries in the segment table of a process = Number of segments that process is divided.
- The base address of the segment table is stored in the segment table base register.

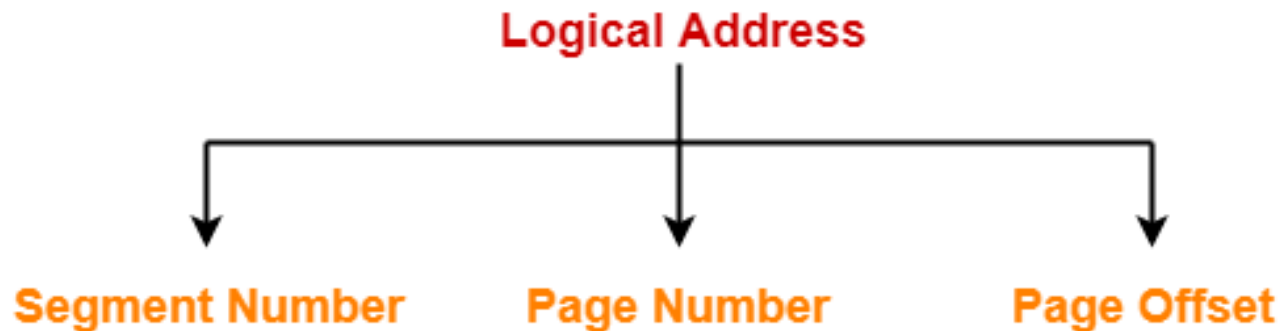
PAGING with SEGMENTATION

- Following steps are followed to translate logical address into physical address-

Step-01:

CPU generates a logical address consisting of three parts-

- Segment Number
- Page Number
- Page Offset





PAGING with SEGMENTATION

Step-02:

- For the generated segment number, corresponding entry is located in the segment table.
- Segment table provides the frame number of the frame storing the page table of the referred segment.
- The frame containing the page table is located.

Step-03:

- For the generated page number, corresponding entry is located in the page table.
- Page table provides the frame number of the frame storing the required page of the referred segment.
- The frame containing the required page is located.

PAGING with SEGMENTATION

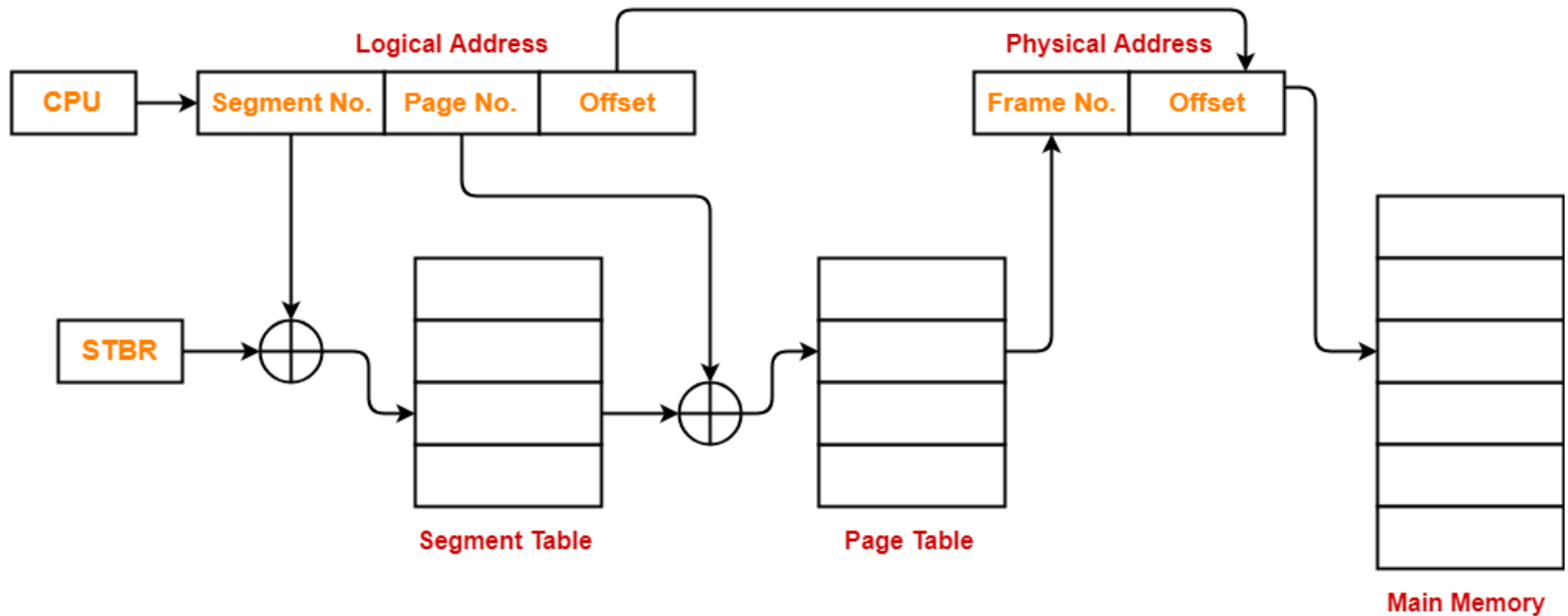
Step-04:

- The frame number combined with the page offset forms the required physical address.
- For the generated page offset, corresponding word is located in the page and read.



PAGING with SEGMENTATION

The following diagram illustrates the above steps of translating logical address into physical address-



Translating Logical Address into Physical Address

PAGING with SEGMENTATION

Advantages-

- Segment table contains only one entry corresponding to each segment.
- It reduces memory usage.
- The size of **Page Table** is limited by the segment size.
- It solves the problem of external fragmentation.

Disadvantages-

- Segmented paging suffers from internal fragmentation.
- The complexity level is much higher as compared to paging.



References

- “Operating System Concepts” by Avi Silberschatz and Peter Galvin
- “Operating Systems: Internals and Design Principles” by William Stallings
- www.tutorialpoint.com
- www.javapoint.com



Thank You

For any Query, you can contact

Er. Inderjeet Singh(e8822)

Ph. No: 86-99-100-160

Email id: inderjeet.e8822@cumail.in